PROJECT DOCUMENTATION REPORT

Credit Card Fraud Detection

A Machine Learning Approach to Classifying Fraudulent Transactions

Prepared by: Satyam Singh

Field: Data Analytics / Machine Learning

Dataset Source: Kaggle — mlg-ulb/creditcardfraud

Report Date: August 2026

Table of Contents

1. Executive Summary

This report documents an end-to-end machine learning project that detects fraudulent credit card transactions. The project was built as a first data science exercise and follows a complete, standard workflow: data acquisition, exploratory data analysis, preprocessing, model training, and evaluation.

Two classification models were trained and compared — Logistic Regression and Random Forest — on a real-world, highly imbalanced dataset of 284,807 anonymized European credit card transactions, of which only 0.17% are fraudulent. The Random Forest model clearly outperformed Logistic Regression, catching 81 of 98 fraud cases in the test set (83% recall) at 94% precision, compared to 63 of 98 (64% recall) at 83% precision for Logistic Regression.

> **Key Takeaway**
> On highly imbalanced data, accuracy is a misleading metric — a model that predicts "legitimate" for every transaction would already be 99.8% accurate. The project correctly shifts focus to precision and recall for the minority (fraud) class.

2. Project Objective

The goal of the project is to build a supervised classification model that predicts whether a given credit card transaction is fraudulent (Class = 1) or legitimate (Class = 0), based solely on transaction-level numerical features.

2.1 Problem Statement

Credit card fraud represents a small fraction of total transactions but causes disproportionate financial harm. A useful fraud-detection model must catch as many fraudulent transactions as possible (high recall) while keeping false alarms on legitimate customers low (high precision) — a classic class-imbalance and precision/recall trade-off problem.

2.2 Success Criteria

- Correctly frame class imbalance and avoid using plain accuracy as the primary metric.
- Train and evaluate at least two different classification algorithms.
- Compare models using precision, recall, and the confusion matrix for the fraud class.
- Select and justify a final ('winning') model.

3. Dataset Description

Attribute	Detail
Source	Kaggle — mlg-ulb/creditcardfraud
Total transactions	284,807
Total features	31 (30 input features + 1 target)
Time period	Transactions made by European cardholders over two days
Target variable	Class (0 = Legitimate, 1 = Fraud)
Fraud cases	492 transactions (0.17% of the dataset)
Legitimate cases	284,315 transactions (99.83% of the dataset)
Missing values	None

3.1 Feature Description

- Time — Seconds elapsed between each transaction and the first transaction in the dataset. Dropped before modeling.
- V1 – V28 — Anonymized, PCA-transformed numerical features (original attributes could not be disclosed for confidentiality).
- Amount — Transaction amount; ranges up to roughly $25,000. Rescaled before modeling.
- Class — Target label: 0 for legitimate, 1 for fraud.

3.2 Descriptive Statistics

- Minimum transaction amount: $0.00
- Mean (average) transaction amount: $88.35
- Median transaction amount (50th percentile): $22.00
- Mean of Class column: 0.001727, confirming a fraud rate of 0.17%

4. Tools and Libraries Used

Library	Purpose
kagglehub	Programmatic download of the dataset directly from Kaggle
pandas	Loading and manipulating the tabular data
matplotlib / seaborn	Data visualization (class-imbalance bar chart, confusion-matrix heatmaps)
scikit-learn (sklearn)	Train/test split, feature scaling, model training, and evaluation metrics

5. Methodology

The notebook follows a linear, step-by-step workflow. Each stage is summarized below along with its purpose and the corresponding logic.

5.1 Step 1 – Import Libraries

The core libraries, kagglehub and pandas, are imported first to enable dataset download and tabular data handling.

```
import kagglehub
import pandas as pd
```

5.2 Step 2 – Download the Dataset

The dataset is downloaded directly from Kaggle using kagglehub, which caches the files outside the project folder.

```
path = kagglehub.dataset_download("mlg-ulb/creditcardfraud")
```

5.3 Step 3 – Load Data into a DataFrame

The CSV file is read into a pandas DataFrame (df), loading all 284,807 transactions into memory.

```
df = pd.read_csv(f"{path}/creditcard.csv")
```

5.4 Step 4 – First Look at the Data

df.head() and df.tail() are used to preview the first and last five rows, confirming the anonymized V1–V28 columns and the Class target column loaded correctly.

```
Data loaded
Rows: 284807
Columns: 31
```

5.5 Step 5 – Inspect the Data

df.info(), df.describe(), and df.isnull().sum() are used to check data types, generate summary statistics, and confirm there are no missing values in the dataset — an important check before any modeling begins.

5.6 Step 6 – Check Class Imbalance

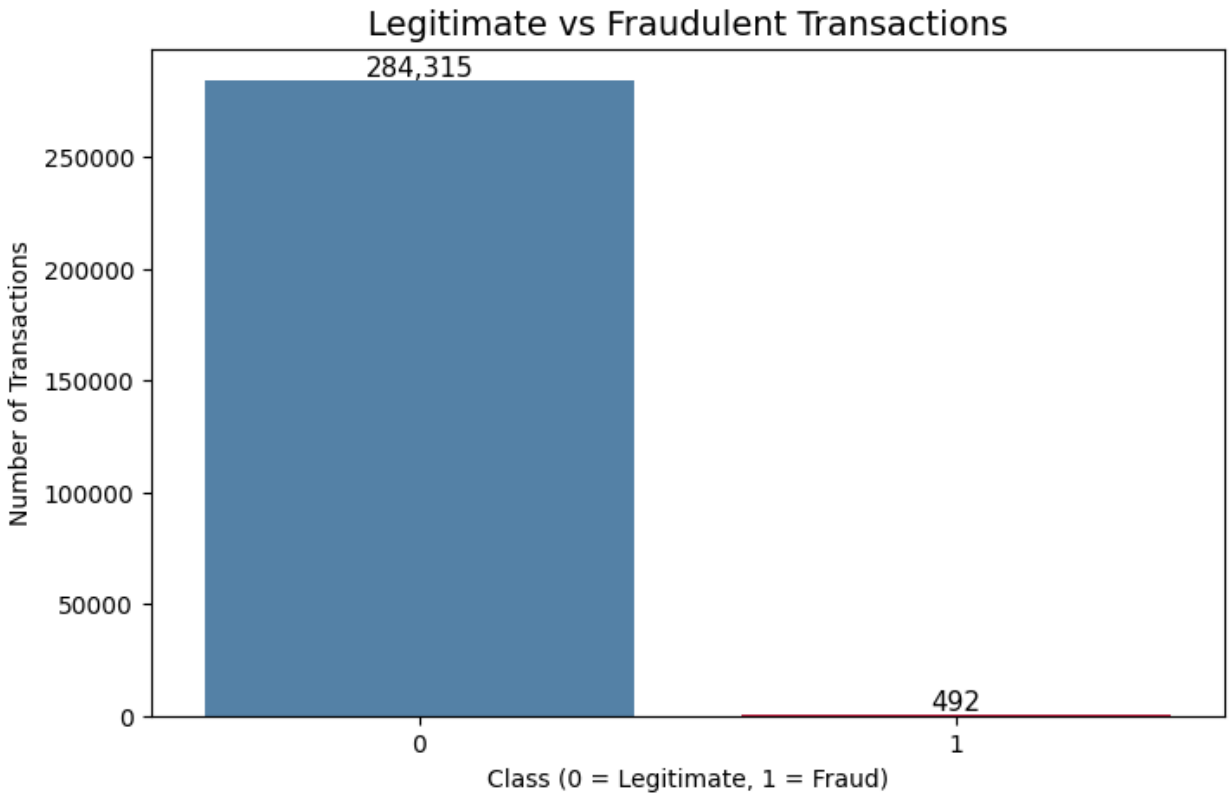
The value counts of the Class column are computed to quantify the imbalance between legitimate and fraudulent transactions.

Class	Count	Percentage
Legitimate (0)	284,315	99.83%
Fraud (1)	492	0.17%

This works out to roughly one fraudulent transaction for every 577 legitimate ones — confirming that this is a severely imbalanced classification problem.

5.7 Step 7 – Visualize the Class Imbalance

A count plot (matplotlib + seaborn) charts the number of transactions per class, with exact counts annotated on top of each bar, making the imbalance visually unmistakable.



5.8 Feature / Target Split

The dataset is split into a feature matrix X (all columns except Class) and a target vector y (the Class column only).

```
X = df.drop('Class', axis=1)
y = df['Class']
```

```
Shape of X (features): (284807, 30)
Shape of y (target):   (284807,)
output actions

First 3 rows of X:
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461

         V8        V9  ...       V20       V21       V22       V23       V24  \
0  0.098698  0.363787  ...  0.251412 -0.018307  0.277838 -0.110474  0.066928
1  0.085102 -0.255425  ... -0.069083 -0.225775 -0.638672  0.101288 -0.339846
2  0.247676 -1.514654  ...  0.524980  0.247998  0.771679  0.909412 -0.689281

        V25       V26       V27       V28  Amount
0  0.128539 -0.189115  0.133558 -0.021053  149.62
1  0.167170  0.125895 -0.008983  0.014724    2.69
2 -0.327642 -0.139097 -0.055353 -0.059752  378.66

[3 rows x 30 columns]

First 3 values of y:
0    0
1    0
2    0
Name: Class, dtype: int64
```

5.9 Train/Test Split

The data is split 80/20 into training and test sets using scikit-learn's train_test_split. Stratified sampling on y preserves the same fraud ratio in both sets, and a fixed random_state ensures reproducibility.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
Training set shape: (227845, 30)
Training answers shape: (227845,)
Testing set shape: (56962, 30)
Testing answers shape: (56962,)
```

5.10 Feature Scaling

The Amount column has a much larger numeric range (up to ~$25,000) than the already-normalized V1–V28 columns, so it is rescaled with StandardScaler. The scaler is fit only on the training data and then applied to the test data, preventing data leakage. The Time column is dropped from both sets since it is not predictive of fraud in this context.

```
scaler = StandardScaler()
X_train['Amount'] = scaler.fit_transform(X_train[['Amount']])
X_test['Amount']  = scaler.transform(X_test[['Amount']])

X_train = X_train.drop('Time', axis=1)
X_test  = X_test.drop('Time', axis=1)
```

6. Model Development

6.1 Model 1 — Logistic Regression

Logistic Regression was chosen as a simple, fast baseline classifier well suited to a first modeling attempt.

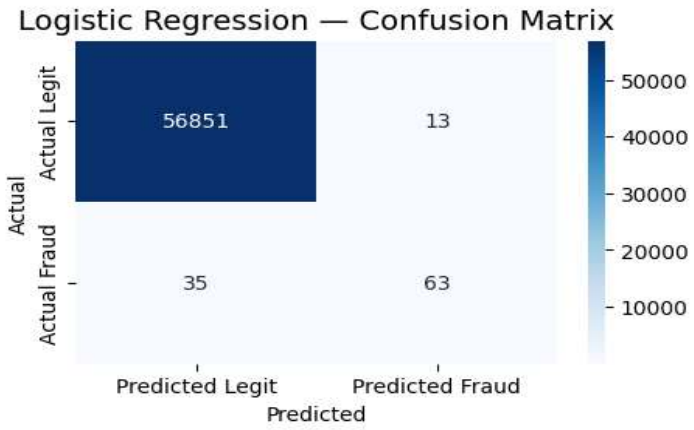
```
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
from sklearn.linear_model import LogisticRegression

# Step 1: Create the model
model = LogisticRegression(max_iter=1000, random_state=42)

# Step 2: Train it on the training data (this is where it learns!)
model.fit(X_train, y_train)

# Step 3: Use the trained model to predict on the test data
y_pred = model.predict(X_test)

print("Model trained successfully!")
print("Number of predictions made:", len(y_pred))
print("Predicted frauds:", (y_pred == 1).sum())
print("Predicted legit:", (y_pred == 0).sum())
```

```
Model trained successfully!
Number of predictions made: 56962
Predicted frauds: 76
Predicted legit: 56886
```

6.1.1 Logistic Regression — Confusion Matrix

Outcome	Count	Meaning
True Negatives	56,851	Correctly identified as legitimate
False Positives	13	Legitimate transactions wrongly flagged as fraud
False Negatives	35	Fraudulent transactions missed
True Positives	63	Fraudulent transactions correctly caught



Confusion Matrix Results

Outcome	Count	What it means
True Negatives	56,851	Correctly said "not fraud" for 56,851 legit transactions
False Positives	13	Wrongly flagged 13 legit transactions as fraud (false alarms, customers get "was this you?" texts)
False Negatives	35	Missed 35 real frauds (fraudster got away, bank loses money)
True Positives	63	Caught 63 real frauds

6.2 Model 2 — Random Forest

Because Logistic Regression caught only 64% of frauds, a Random Forest Classifier — an ensemble of 100 decision trees — was trained as a stronger alternative capable of capturing more complex, non-linear patterns.

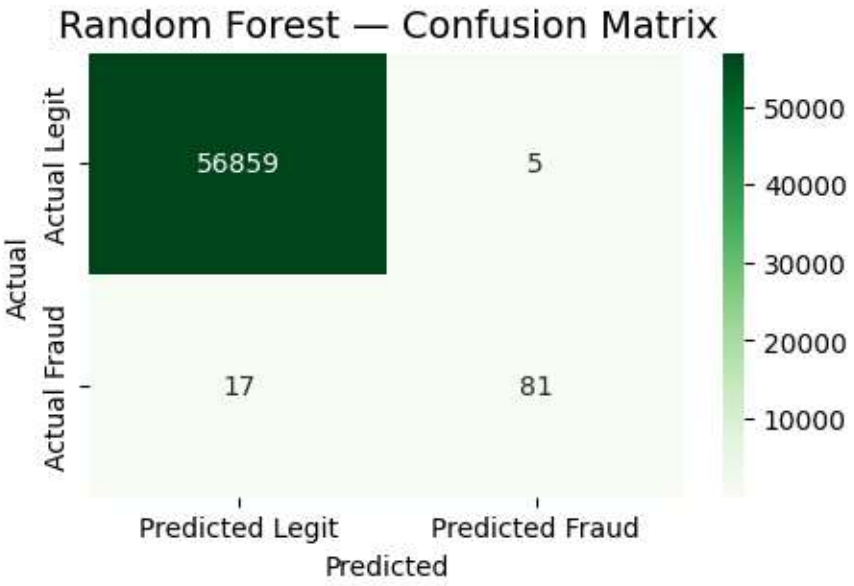
```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
```

```
Training Random Forest...
Training done!

Confusion Matrix:
[[56859     5]
 [   17    81]]

Classification Report:
              precision    recall  f1-score   support

   Legit (0)       1.00      1.00      1.00     56864
   Fraud (1)       0.94      0.83      0.88        98

    accuracy                           1.00     56962
   macro avg       0.97      0.91      0.94     56962
weighted avg       1.00      1.00      1.00     56962
```

6.2.1 Random Forest — Confusion Matrix

Outcome	Count	Meaning
True Negatives	56,859	Correctly identified as legitimate
False Positives	5	Legitimate transactions wrongly flagged as fraud
False Negatives	17	Fraudulent transactions missed
True Positives	81	Fraudulent transactions correctly caught

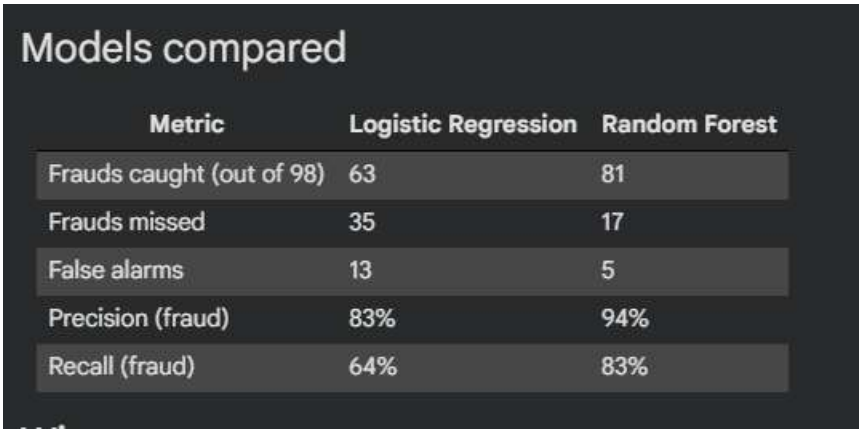
```
Training Random Forest...
Training done!

Confusion Matrix:
[[56859     5]
 [   17    81]]

Classification Report:
              precision    recall  f1-score   support

   Legit (0)       1.00      1.00      1.00     56864
   Fraud (1)       0.94      0.83      0.88        98

    accuracy                           1.00     56962
   macro avg       0.97      0.91      0.94     56962
weighted avg       1.00      1.00      1.00     56962
```



7. Results and Model Comparison

Both models were evaluated on the same held-out test set of 56,962 transactions (98 of them fraudulent), using precision, recall, and the confusion matrix for the fraud class rather than raw accuracy.

Metric	Logistic Regression	Random Forest
Frauds caught (out of 98)	63	81
Frauds missed	35	17
False alarms	13	5
Precision (fraud class)	83%	94%
Recall (fraud class)	64%	83%



7.1 Interpretation

- Random Forest improved recall from 64% to 83% — catching 18 more of the 98 fraud cases in the test set.
- Random Forest also improved precision from 83% to 94%, while simultaneously reducing false alarms from 13 to 5.
- Random Forest dominates Logistic Regression on every reported metric — it is a strict improvement, not a trade-off.

8. Conclusion

This project built a working fraud-detection pipeline on a real, highly imbalanced dataset of 284,807 transactions. Because only 0.17% of transactions are fraudulent, accuracy alone would be a misleading measure of success, so evaluation was correctly centered on precision and recall for the fraud class.

Two models were compared: Logistic Regression as a fast, interpretable baseline, and Random Forest as a stronger ensemble method. Random Forest was selected as the final model, catching 81 of 98 fraud cases (83% recall) at 94% precision, clearly outperforming Logistic Regression's 63 of 98 (64% recall) at 83% precision.

> **Winner: Random Forest**
> Random Forest caught more frauds while generating fewer false alarms, making it the more reliable choice for a real fraud-detection use case.

9. Limitations and Future Improvements

9.1 Limitations

- The features V1–V28 are PCA-anonymized, so there is no way to interpret which real-world transaction attributes drive predictions.
- No resampling techniques (e.g. SMOTE, undersampling) or class-weighting were applied — both models were trained directly on the imbalanced data.
- Only default / lightly-tuned hyperparameters were used for both models; no cross-validation or hyperparameter search was performed.
- Threshold tuning (adjusting the classification cutoff away from 0.5) was not explored, even though it could further improve the recall/precision trade-off.

9.2 Suggested Next Steps

- Apply SMOTE or class-weight balancing to address the class imbalance more directly.
- Perform hyperparameter tuning (e.g. GridSearchCV / RandomizedSearchCV) on the Random Forest model.
- Evaluate additional metrics such as ROC-AUC and Precision-Recall AUC, which are more informative than accuracy on imbalanced data.
- Test additional algorithms such as XGBoost or LightGBM, which often perform strongly on tabular fraud-detection tasks.
- Explore threshold tuning to adjust the precision/recall balance based on business cost of false positives vs. false negatives.

10. Appendix — Full Workflow Reference

Step	Notebook Section	Purpose
1	Import Libraries	Load kagglehub and pandas
2	Download the Dataset	Fetch dataset from Kaggle
3	Load Data into a DataFrame	Read CSV into pandas

Step	Notebook Section	Purpose
4	First Look at the Data	Preview head() / tail()
5	Inspect the Data	info(), describe(), null check
6	Check Class Imbalance	value_counts() on Class
7	Visualize Class Imbalance	Seaborn count plot
8	Feature/Target Split	X and y separation
9	Train/Test Split	80/20 stratified split
10	Feature Scaling	StandardScaler on Amount, drop Time
11	Model 1: Logistic Regression	Baseline model + evaluation
12	Model 2: Random Forest	Ensemble model + evaluation
13	Conclusion	Comparison and final model choice